

ENTRE MUNDOS: DESENVOLVIMENTO DE UM *ACTION RPG* COM GERAÇÃO PROCEDURAL EM PYTHON E PYGAME

Wesley Fernandes Rocha*

Anhanguera

18 de maio de 2026

Resumo

Este artigo descreve o processo de desenvolvimento do jogo eletrônico “Entre Mundos - RPG Ultimate”, um *Action Role-Playing Game* (RPG) de plataforma bidimensional construído integralmente com a linguagem Python e a biblioteca Pygame. O projeto teve como principal objetivo aplicar e consolidar conceitos fundamentais de Engenharia de Software, tais como Programação Orientada a Objetos (POO), arquitetura de laços de repetição em tempo real (*Game Loop*), e máquinas de estado. Destaca-se no projeto a utilização de renderização procedural de entidades, substituindo imagens estáticas por polígonos calculados matematicamente em tempo real, além da implementação de sistemas dinâmicos de física e interações de cenário, como o sistema de montarias. Os resultados demonstram que a construção de motores de jogo em ambientes acadêmicos é uma ferramenta eficaz para o domínio de otimização de processamento e lógica estruturada.

Palavras-chave: Desenvolvimento de Jogos. Python. Pygame. Geração Procedural. Engenharia de Software.

*Estudante de análise e desenvolvimento de sistemas na anhanguera.

1 INTRODUÇÃO

O desenvolvimento de jogos eletrônicos é amplamente reconhecido como uma das áreas mais complexas e multidisciplinares da Ciência da Computação. Diferente de softwares tradicionais baseados em eventos passivos de interface de usuário, um jogo exige um ciclo de atualização contínuo (*Game Loop*) capaz de processar entradas físicas, calcular colisões, gerenciar memória e renderizar gráficos dezenas de vezes por segundo, tudo sob rigorosas restrições de tempo [1].

Neste contexto acadêmico, o projeto “Entre Mundos - RPG Ultimate” foi concebido como um campo de testes prático para a aplicação de conceitos de Programação Orientada a Objetos (POO) e otimização de algoritmos. O jogo apresenta uma experiência de plataforma de ação bidimensional, estruturada em múltiplas classes de personagens (Guerreiro, Mago e Arqueiro), progressão de níveis, e uma variedade de biomas com comportamentos de inteligência artificial específicos para inimigos e chefes.

O objetivo deste artigo é detalhar a arquitetura técnica que suporta o jogo, focando nas decisões de design de software que permitiram a criação de um motor físico próprio, a gestão de estados dinâmicos (como a integração de montarias interativas) e a opção pela renderização visual através de arte procedural matemática em detrimento do uso massivo de *sprites* pré-renderizados.

2 METODOLOGIA E FERRAMENTAS

A linguagem de programação selecionada para o projeto foi o Python, devido à sua sintaxe clara, excelente suporte ao paradigma de Orientação a Objetos e vasta adoção na comunidade acadêmica. Como base gráfica e de interface, optou-se pela biblioteca *Pygame*, que fornece abstrações diretas (primitivas de desenho) sobre o *Simple DirectMedia Layer* (SDL), permitindo acesso de baixo nível aos periféricos do sistema operativo (teclado, rato e áudio) sem ofuscar a necessidade de programar a lógica fundamental [2].

O código foi estruturado em um monólito modular, onde o arquivo principal organiza a importação de dependências e declara as classes de forma hierárquica. As principais entidades do jogo (Jogador, Inimigos, Projéteis e Cenários) herdam da classe `pygame.sprite.Sprite`, facilitando o agrupamento e a detecção de colisões.

3 ARQUITETURA E DESENVOLVIMENTO

O motor do jogo foi desenvolvido para resolver problemas clássicos de simulação física e renderização gráfica computacional, estruturando o comportamento das entidades através de regras bem definidas.

3.1 Física e Tratamento de Colisões

A movimentação da entidade principal (Player) transcende a simples alteração de coordenadas estáticas. O motor implementa vetores de velocidade horizontal (v_x) e vertical (v_y), sobre os quais atua uma constante gravitacional global. A resolução de colisões utiliza o teorema de separação de eixos (AABB - *Axis-Aligned Bounding Box*), processando os eixos de forma independente. O jogador move-se primeiramente no eixo horizontal, verificando interseções com a lista de plataformas e ajustando a sua posição para a borda correspondente. Em seguida, o processo é repetido para o eixo vertical.

Esta abordagem independente previne *bugs* comuns na física de jogos, como a travessia acidental de superfícies sólidas em altas velocidades (*tunneling*), e permite a implementação fluida de mecânicas responsivas.

3.2 Renderização Procedural e Matemática

Uma das características arquiteturais mais relevantes de “Entre Mundos” é o uso de renderização procedural. Em vez de depender de arquivos externos de imagem (*.png* ou *.jpg*), a maioria dos modelos de heróis, chefes e efeitos visuais são desenhados em tempo real utilizando funções de primitivas geométricas (`pygame.draw.polygon`, `pygame.draw.circle`).

Além de reduzir drasticamente o tamanho em disco do software e o tempo de carregamento (I/O), a renderização por código permite animações controladas matematicamente. Através da aplicação de funções trigonométricas de seno e cosseno atreladas ao *clock* do sistema (`pygame.time.get_ticks()`), implementou-se oscilações fluídas (*bob* e *swing*) que conferem uma sensação de “respiração” e peso aos personagens, independentemente da taxa de quadros (FPS) em que o jogo esteja sendo executado.

3.3 Padrão de Estados: Sistema de Montaria

O desenvolvimento do sistema de montarias evidencia a aplicação prática de Máquinas de Estado (*State Machine*). O fluxo inicia-se com um objeto estático, a Relíquia Anciã (BlocoBrilhante), que pulsa através da variação procedural de

valores RGB. Quando o jogador (em um estado específico de colisão e ataque) interage com a relíquia, ela é destruída da memória via método `kill()` e instancia o objeto `Montaria`.

Ao colidir com a montaria ativa, o estado da classe `Player` é alterado (via *flag* booleana `is_mounted = True`). Esta simples alteração de estado reverbera por múltiplos sistemas da classe:

- **Renderização:** O método `draw` passa a renderizar a besta mítica abaixo do jogador, deslocando os *pixels* do herói para cima.
- **Física:** Os limitadores de velocidade e a força do pulo são multiplicados por matrizes de modificação.
- **Sistema de Dano (*Hitpoints*):** O método `take_damage` verifica o estado. Se montado, a montaria absorve o impacto (reduzindo o dano global) e é descartada, retornando o jogador ao estado padrão, provando a eficácia de uma lógica coesa de controle de status.

4 RESULTADOS OBTIDOS

O jogo, executando o seu laço principal de atualização e desenho, provou ser altamente estável. O processamento sequencial de até dezenas de inimigos, juntamente com o gerenciamento autônomo de vetores e centenas de instâncias do sistema de partículas procedurais, executou de forma fluida sem quedas drásticas de processamento (*frame drops*) no ambiente de testes.

A restrição auto imposta de desenhar modelos complexos utilizando apenas polígonos computacionais evidenciou uma profunda compreensão sobre a manipulação de matrizes de superfície (*Surfaces*) e uso de canal alfa (*Alpha Blending*) em Pygame. A experiência de utilizador foi complementada com recursos de *Game Feel*, como *screen shake* e pausas de impacto (*hitstop*) de milissegundos após golpes críticos, melhorando drasticamente o peso visual da interação.

5 CONSIDERAÇÕES FINAIS

O projeto “Entre Mundos - RPG Ultimate” atingiu o seu objetivo primordial como ferramenta de pesquisa empírica em Engenharia de Software focada em simulações em tempo real. A implementação direta da física limitadora, sem recorrer a motores proprietários abstratos (como *Unity* ou *Godot*), exigiu uma profunda absorção da álgebra linear vetorial e do planejamento estrito de herança de classes.

Para trabalhos e atualizações futuras, sugere-se a refatoração do código monolítico para o padrão de arquitetura *Model-View-Controller* (MVC), bem como a introdução de concorrência com o módulo `threading` nativo do Python, com vista à comunicação via soquetes (*Sockets*) para o desenvolvimento do módulo *multi-player*.

REFERÊNCIAS

- [1] NYSTROM, R. **Game Programming Patterns**. [S.l.]: Genndy, 2014. Disponível em: <<https://gameprogrammingpatterns.com/>>.
- [2] PYGAME. **Pygame Documentation: Pygame v2.6.1**. Disponível em: <<https://www.pygame.org/docs/>>. Acesso em: 18 maio 2026.
- [3] PYTHON SOFTWARE FOUNDATION. **The Python Language Reference, version 3.12**. Disponível em: <<https://docs.python.org/3/reference/index.html>>. Acesso em: 18 maio 2026.
- [4] SOMMERVILLE, I. **Engenharia de Software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.